Prompt Engineering for Pedagogical Content Generation

CSE 329: Machine Learning | Total: 100 marks | Prompt Quality 50% + Content Quality 50%

1. Overview

In this individual assignment, you will design a modular prompt system that uses an LLM to generate pedagogical materials on your assigned ML topic. Your topic is determined by your student ID (see the separate Topic Assignment document: Topic # = (Student ID mod 25) + 1). You will submit three prompt-generated outputs (each reviewed via a Human-in-the-Loop process), the prompts themselves, and a review log. Your final score combines absolute rubric quality (60%) with relative ranking within your topic cohort (40%)—see the Topic Assignment document for details.

What You Will Produce

Deliverable A — Modular Prompt System: A system prompt plus three module prompts (see pages 2–4), submitted as .md files.

Deliverable B — Generated Content (HITL-Edited): The final, human-reviewed output from each module: lecture note, theory assessment, and coding files.

Deliverable C — HITL Review Log: A structured record of raw LLM output, issues found, corrections made, and rationale for each change.

Submission Format

```
StudentID_TopicName/
├── prompts/          system_prompt.md, module1-module3.md
├── outputs/          lecture_note.md, theory_assessment.md,
│                    coding_boilerplate.py, coding_solution.py
├── hitl_log/         hitl_review_log.md
└── README.md          Name, student ID, topic, LLM used, approach summary
```

Key Concepts You Need

Bloom's Taxonomy: A six-level hierarchy of cognitive skills—Remember, Understand, Apply, Analyze, Evaluate, Create—used to design assessment questions at varying depths. See the reference table on page 3.

Constructive Alignment: A design principle requiring that learning objectives, content, and assessment all target the same skills. If the lecture note teaches concept X, the assessment must test X, and the code must require implementing X.

Human-in-the-Loop (HITL): Every LLM output must pass through documented human review. Raw, unreviewed output is not acceptable. Your HITL log must show: raw output → issues identified → corrections → rationale.

Prompt Design Expectations

Your prompt system must be modular (system prompt + 3 independent module prompts), topic-agnostic (reusable by changing one variable), and employ at least 4 deliberate PE techniques (role prompting, chain-of-thought, few-shot examples, output structuring, constraint setting, iterative refinement, or meta-prompting). A separate grading rubric is provided.

2. Module 1 — Lecture Note

Generate a detailed, pedagogically sound lecture note on your assigned topic from the class lectures of all sections of CSE 329 as well as any additional topic necessary to understand taught content. The note must contain three clearly separated components.

A) Knowledge Component

Covers the theoretical foundations students need to understand the topic.

- Core concepts, mathematical foundations, and underlying theory.
- Relevant formulas, equations, and derivations with clear notation.
- Step-by-step algorithmic descriptions (pseudocode or structured prose).
- Comparison with related methods: strengths, weaknesses, and when to use each.
- Real-world application examples with concrete use cases.

B) Skill Component

Equips students with the practical ability to implement the technique.

- Step-by-step implementation guide a student could follow independently.
- Code examples in Python using standard ML libraries (scikit-learn, PyTorch, TensorFlow, etc.).
- End-to-end workflow: data preprocessing → model training → evaluation.
- Common pitfalls, debugging tips, and performance tuning advice.
- Hyperparameter tuning guidelines with recommended search ranges and strategies.

C) Ethical Implications & Values Component

Ensures students engage with the responsible use of the technique.

- Potential biases introduced or amplified by this technique and how to detect them.
- Privacy and data protection concerns specific to this method.
- Societal impact: who benefits, who may be harmed, and under what conditions.
- Responsible AI practices applicable to this technique.
- At least one concrete ethical scenario or case study tied to the topic.

> *Quality check: The three components must be internally consistent and collectively cover the topic at a depth appropriate for upper-undergraduate ML students. The ethics component must be substantive and topic-specific—not generic AI ethics boilerplate.*

3. Module 2 — Theory Assessment

Generate a theory assessment with questions at all six Bloom's Taxonomy levels, plus complete answer keys for the produced lecture notes.

Level	Cognitive Skill	What it means	Action Verbs	ML Example
1. Remember	Recall facts from memory	Student retrieves definitions, formulas, or terms without deep understanding.	Define, list, state, recall, identify	*"State the ELBO formula."*
2. Understand	Explain in own words	Student paraphrases, summarizes, or explains why something works.	Explain, compare, summarize, interpret	*"Explain why reparameterization enables backprop in VAEs."*
3. Apply	Use in a new situation	Student uses a learned procedure to solve a problem they haven't seen before.	Calculate, implement, solve, predict	*"Calculate information gain for this candidate split."*
4. Analyze	Examine relationships	Student decomposes a system, identifies patterns, or diagnoses failures.	Differentiate, contrast, diagnose, examine	*"Why does this GNN fail on heterophilic graphs?"*
5. Evaluate	Justify a judgment	Student assesses merit, critiques a choice, or defends a position with evidence.	Justify, critique, assess, argue, recommend	*"Evaluate choosing LIME over SHAP for fraud detection."*
6. Create	Produce original work	Student designs a novel solution, proposes an experiment, or synthesizes across areas.	Design, propose, formulate, synthesize	*"Design a federated protocol addressing heterogeneity and privacy."*

Assessment Requirements

1. At least 2 questions per Bloom's level (minimum 12 questions total).
2. Each question must be labeled with its Bloom's level and the target action verb.
3. Questions at levels 4–6 must present novel scenarios—not lecture note examples restated.
4. Each answer key must include: model answer, marking scheme (partial credit rules), and expected response depth.
5. At least one question at Evaluate or Create level must engage with an ethical dimension.

> *Common mistake: Labeling a recall question as "Analyze." True analysis requires decomposing a problem or diagnosing a failure—not restating facts. Use the action verbs column as a self-check.*

4. Module 3 — Coding Assessment

Generate two Python files: a student-facing boilerplate and a complete instructor solution. Both must be runnable .py scripts (not notebook fragments). Specify Python version and all library versions in the file header.

A) Student Boilerplate

Pre-completed (students don't touch): All imports with version checks, a config block (hyperparameters, paths, seeds—no magic numbers), data loading/preprocessing/splitting, logging/plotting utilities, and a main() entry point.

TODO sections (students implement): 4–8 tasks progressing in difficulty, each following this format:

```
# TODO [T1]: [Short title]
# Bloom's Level: [Apply / Analyze / Create]
# Difficulty: [Easy / Medium / Hard]  |  Expected lines: ~N
# Description: [2-4 sentences: what to implement and why]
# Hints: 1. [conceptual]  2. [implementation-specific]
# Expected behavior: [output shape, value range, or result]
# >>> YOUR CODE HERE <<<
raise NotImplementedError("TODO T1 not yet implemented")
# >>> END YOUR CODE <<<
```

TODO Design Rules

- At least one TODO at Apply, one at Analyze, and one at Create level.
- Every TODO must have a unique ID (T1–T8) and a raise NotImplementedError.
- Include a validation harness: a function students can call to check their work.

B) Instructor Solution

- Every TODO filled with correct, working code, preceded by a comment explaining the approach.
- Runs end-to-end without errors on a specified dataset (named in the file header).
- Documents expected outputs: printed metrics, loss ranges, reference plots.
- Includes performance benchmarks: training time, final metric values, hardware assumptions.

C) Pre-Submission Checklist

Check	Boilerplate	Solution
Runs without errors	TODOs raise NotImplementedError	Runs end-to-end
Imports present & version-pinned	Required	Required
Config block (no magic numbers)	Required	Required
TODO format matches spec above	Required	N/A (filled in)
Validation harness included	Required	Passes all checks
Comments explain "why"	In TODO descriptions	In solution comments

5. HITL Process, Academic Integrity & Tips

5.1 HITL Workflow (Mandatory)

Every module's output must pass through this review before submission:

Step	Activity	Output
1	Design prompts and generate LLM output	Raw output
2	Verify technical accuracy (equations, algorithms, code correctness)	Annotated accuracy notes
3	Review pedagogical quality (Bloom's level accuracy, constructive alignment)	Annotated pedagogy notes
4	Test all code (boilerplate runs, solution produces expected output)	Test results
5	Apply corrections, re-run prompts if needed, finalize all outputs	Final output + HITL log

5.2 HITL Review Checklist

- Technical accuracy: equations, algorithms, and code correct?
- Completeness: all required components present across all three modules?
- Bloom's alignment: are theory questions genuinely at their labeled cognitive level?
- Code functionality: boilerplate runs (TODOs raise errors), solution runs end-to-end?
- Cross-module consistency: do lecture note, assessment, and code all align?
- Ethical coverage: substantive, topic-specific analysis, not an afterthought?

5.3 Academic Integrity

- Prompt design must be original. Copying from another student is a violation.
- LLM-generated content is permitted (that is the assignment)—but only after genuine HITL review.
- Fabricating a HITL log (writing it after the fact without actually reviewing) is a violation.
- Disclose the LLM model, version, and platform in your README.
- All code must actually run. Non-functional code that merely looks correct is not acceptable.

5.4 Tips for Success

- Invest time in prompt design. Better prompts = better output = less HITL editing.
- Start small—test prompts on a single section before generating the full output.
- Work through the HITL steps in order: accuracy first, then pedagogy, then code testing. Don't skip steps.
- Use few-shot examples in prompts. Even one example dramatically improves quality.
- Always run the code. LLMs produce subtle bugs (wrong shapes, deprecated calls). Test everything.
- Check Bloom's levels carefully. An important error is labeling recall questions as analysis—use the action verbs table on page 3.
- For ethics, avoid platitudes. Tie analysis to specific properties of your technique.

Good luck — build something you'd actually want to learn from.

Grading rubric provided separately. See Topic Assignment document for your topic and grading formula.